

Product Catalogue Application

Associate Flutter Developer Assessment

A modern, responsive, and robust Flutter Product Catalogue application built for the Associate Flutter Developer practical assessment. The application features real-time search, category filtering, persistent favourites, light/dark Material 3 themes, API data fetching with offline fallback, hero transitions, and comprehensive state handling (loading, error with retry, and empty states).

Features

- **Product List Screen:** Responsive grid layout (2–4 columns depending on screen size) showing product images, title, price, category badges, rating indicators, and interactive heart favourite buttons.
- **Product Details Screen:** High-resolution Hero image transition, full product breakdown, rating statistics, expanded description, and synchronized favourite toggle.
- **Real-Time Substring Search:** Live search field with instant substring matching across product titles and categories, with a clear button.
- **Category Filter Chips:** Horizontally scrollable chip filter (All, Men's Clothing, Jewellery, Electronics, etc.).
- **Favourites Management & Persistence:** Toggle favourites on either screen with state synchronized instantly across both screens. Favourites are persisted across app restarts using `shared_preferences`.
- **Light & Dark Material 3 Themes:** Persistent light and dark themes with custom color palettes, accessible via a top app bar toggle button.
- **Graceful Error & Loading States:** Progress indicators during fetch, clear error states with an interactive Retry button for network failures, and empty states for unmatched searches or empty favourites.

Setup & Execution Instructions

Prerequisites

- Flutter SDK (v3.19.0 or later)
- Dart SDK (v3.3.0 or later)
- Android Studio / VS Code with Flutter extension
- Java Development Kit (JDK 17 or higher for Android builds)

1. Install Dependencies

Clone or download the repository, navigate to the root folder, and fetch the package dependencies:

```
flutter pub get
```

2. Run the Application

To launch the app on a connected emulator or physical device:

```
flutter run
```

To run on desktop (Windows):

```
flutter run -d windows
```

To run on Chrome (Web):

```
flutter run -d chrome
```

3. Run Unit & Widget Tests

Execute the unit and widget test suite:

```
flutter test
```

4. Build Android APK

To compile a release APK for distribution:

```
flutter build apk --release
```

The output APK file will be located at:

```
build/app/outputs/flutter-apk/app-release.apk
```

Architecture & Project Structure

The project follows a Feature-Layered Clean Architecture pattern ensuring separation of concerns, testability, and high maintainability.

```
lib/
├── core/
│   ├── theme/
│   │   ├── app_theme.dart          # Light and Dark ThemeData (Material 3)
│   │   └── theme_provider.dart      # Theme mode state notifier with persistence
│   └── utils/
│       └── formatters.dart          # String and currency formatting tools
├── data/
│   ├── models/
│   │   └── product.dart             # Product entity with JSON deserialization & copyWith
│   ├── repositories/
│   │   └── product_repository.dart  # Repository combining API fetching and offline mock
├── fallback
│   └── services/
│       ├── api_service.dart         # HTTP service targeting FakeStore API
│       └── storage_service.dart      # SharedPreferences service for favourites & theme
├── storage
├── presentation/
│   ├── providers/
│   │   └── product_provider.dart    # Main application state notifier (search, filters,
favourites)
│   └── screens/
│       └── product_list_screen.dart  # Main grid screen with app bar controls and filter
bars
│   └── product_detail_screen.dart    # Detailed product view with Hero animation
├── widgets/
│   ├── product_card.dart            # Card widget displaying product image, price,
rating, category
│   ├── product_search_bar.dart      # Input field with substring search & clear action
│   ├── category_filter_bar.dart      # Horizontal ChoiceChip filter list
│   ├── empty_state_widget.dart      # Custom illustration/message when no results found
│   └── error_state_widget.dart       # Error display widget featuring a Retry button
└── main.dart                        # Entry point initializing dependencies & MultiProvider
```

State Management Approach

- Uses `provider` (ChangeNotifier + MultiProvider) as the primary state management solution.
- `ProductProvider` holds product catalog state, search query, selected category filter, and persistent favourite IDs.
- `ThemeProvider` manages the active `ThemeMode` (light vs dark) and persists user preference.

API Integration & Data Handling

- **Remote API:** Fetches product data from FakeStore API (<https://fakestoreapi.com/products>) using `http.Client`.
- **Resilience / Offline Fallback:** If the network is unavailable or times out, `ProductRepository` seamlessly falls back to a realistic local mock dataset. This ensures zero app crashes and full functionality even without an active internet connection.

Assumptions Made

1. **FakeStore API Availability:** Assumed FakeStore API (<https://fakestoreapi.com/products>) as a representative standard e-commerce endpoint, complemented with a mock fallback for offline resilience.
2. **Local Persistence:** Assumed `SharedPreferences` is sufficient for persisting lightweight data (favourite product IDs and theme selection).
3. **Screen Responsiveness:** Assumed the app will be viewed across mobile devices and desktop windows; implemented responsive grid column counts accordingly (2 columns on mobile, 3–4 on wider screens).

Challenges & Solutions

Challenge	Solution
Network Flakiness / Offline Reliability	Implemented a fallback mechanism inside <code>ProductRepository</code> that automatically supplies mock products if HTTP requests time out or fail.
State Synchronization Across Screens	Centralized favourite management in <code>ProductProvider</code> using a <code>Set<int></code> of favourite IDs. Toggling favourites on either the list card or detail screen immediately triggers <code>notifyListeners()</code> , keeping both UI views in sync.
Handling Image Loading & Failures	Wrapped product images in <code>Image.network</code> with custom <code>loadingBuilder</code> and <code>errorBuilder</code> to display smooth progress indicators and broken-image fallbacks gracefully.

Future Improvements

- **Pagination / Infinite Scroll:** Implement lazy loading for larger product catalogs.
- **Cart & Checkout Flow:** Add shopping cart state management and checkout workflow.
- **Caching Images:** Integrate `cached_network_image` for offline image caching.
- **Sort Options:** Add sorting options by price (low to high, high to low) and rating.